Neural Networks

COMP4153 Natural Language Processing

Kian Ming Lim

School of Computer Science
University of Nottingham Ningbo China

Learning outcomes

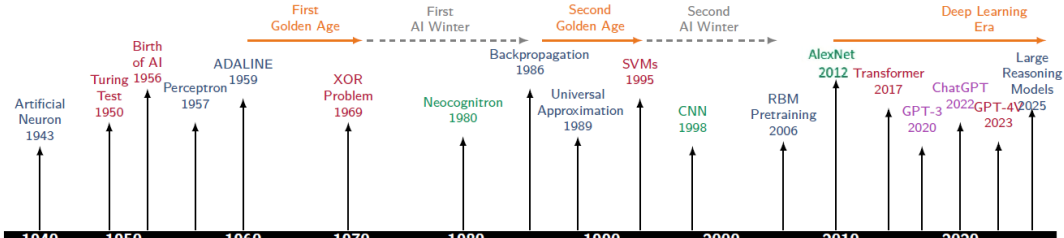
By the end of this lecture, you should be able to:

- explain the perceptron as a simple neural classifier;
- describe why nonlinearity cannot be solved by a single linear decision boundary;
- explain a multilayer perceptron for a basic NLP task;
- explain practical issues such as activations, loss functions, learning rate and overfitting.

Outline

1. Historical Motivation
2. Perceptron
3. Neural Networks
4. Forward Pass
5. Loss and Training
6. Backward Pass
7. Multi-class Neural Networks
8. Practical considerations
9. Summary

Timeline of Neural Networks and Deep Learning



McCulloch–Pitts logic neuron



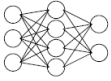
Rosenblatt single-layer learning



Minsky–Papert limits of perceptrons

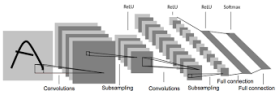


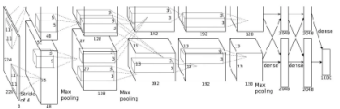
Rumelhart, Hinton and Williams



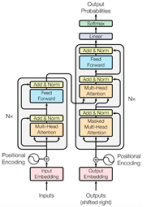
LeCun CNNs

Krizhevsky et al. deep CNNs

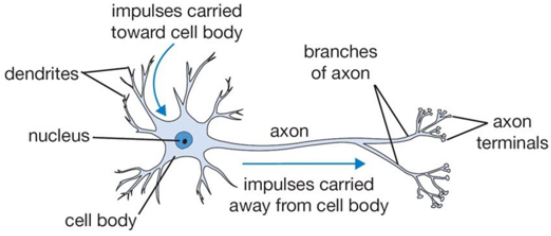


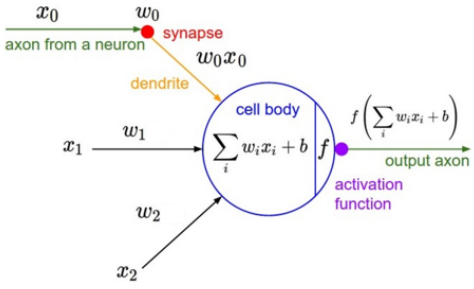


Vaswani et al. attention



Biological Neuron and Artificial Neuron



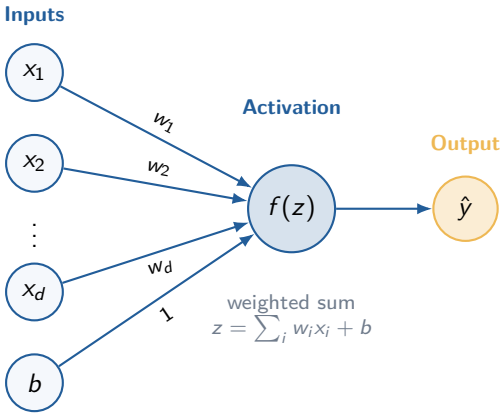


Outline

1. Historical Motivation
2. Perceptron
3. Neural Networks
4. Forward Pass
5. Loss and Training
6. Backward Pass
7. Multi-class Neural Networks
8. Practical considerations
9. Summary

Perceptron

- A perceptron is a simple linear threshold neuron/classifier; a single-layer perceptron network contains no hidden layer.
- First studied in the 1950s.
- Other neural networks were known, but the perceptron attracted attention because it was capable of **learning from data**.
- This made it the main focus of early neural network research.



$$z = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$$

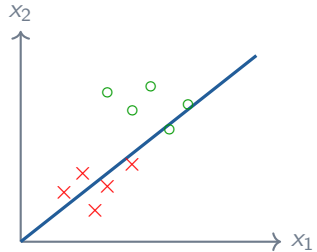
$$\hat{y} = f(z)$$

Why Do We Need the Bias Term?

- The bias term, b, allows a linear model to have a decision boundary that is **not fixed at the origin**.
- Without b, the boundary must pass through the origin, which may give a poor fit.
- With b, the perceptron can shift the decision boundary to separate the classes better.

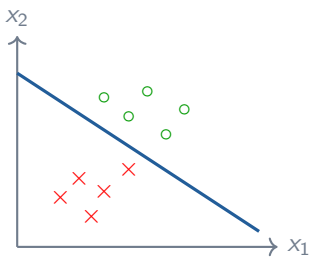
$$z = w_1 x_1 + w_2 x_2 + b$$

Without bias: $b = 0$



Boundary is forced to pass through the origin.

With bias: $b \neq 0$



Bias shifts the boundary for a better separation.

Bias as an Extra Weight

- The bias term can be written as an additional weight.
- Define a constant input:

$$x_0 = 1$$

- Then set:

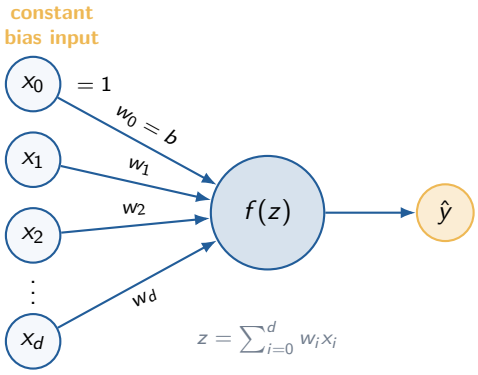
$$w_0 = b$$

- So the perceptron equation becomes:

$$z = w_0 x_0 + w_1 x_1 + \cdots + w_d x_d$$

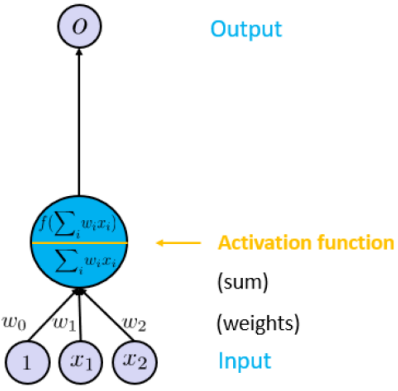
- This allows us to write the model compactly as:

$$z = \sum_{i=0}^{d} w_i x_i$$



Key idea: the bias is treated like a normal weight connected to a constant input.

Perceptron



Given input features $\{x_i\}$, the target label is:

$$y \in \{0, 1\}$$

The perceptron first computes a score:

$$z = w_0 + w_1 x_1 + \cdots + w_d x_d = \boldsymbol{w}^\top \boldsymbol{x}$$

where

$$\boldsymbol{x} = [1, x_1, \ldots, x_d]^\top, \qquad \boldsymbol{w} = [w_0, w_1, \ldots, w_d]^\top.$$

It then predicts a hard class label:

$$\hat{y} = f(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

Notation: y is the true label; $\hat{y}$ is the predicted label.

Perceptron Learning Algorithm

Goal: learn weights that correctly classify training examples.

$$(\boldsymbol{x}, y), \quad y \in \{0, 1\}, \qquad \boldsymbol{x} = [1, x_1, \ldots, x_d]^\top$$

Perceptron Training Procedure

1. Initialise the weight vector:
$$\boldsymbol{w} = [w_0, w_1, \ldots, w_d]^\top$$
2. For each training example, compute the score:
$$z = \boldsymbol{w}^\top \boldsymbol{x}$$
3. Predict the class label:
$$\hat{y} = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$
4. If the prediction is wrong, update:
$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta\big(y - \hat{y}\big)\boldsymbol{x}$$

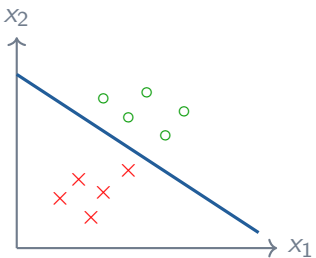
What the perceptron can and cannot do

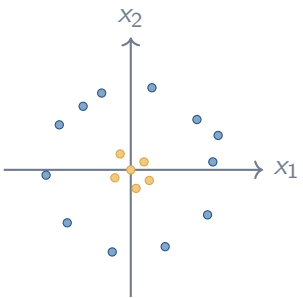
Can learn:

- simple linear separation;
- useful feature weights;
- fast baseline classifiers.

Cannot learn:

- nonlinear decision boundaries;
- feature interactions automatically;
- hidden representations from raw inputs.



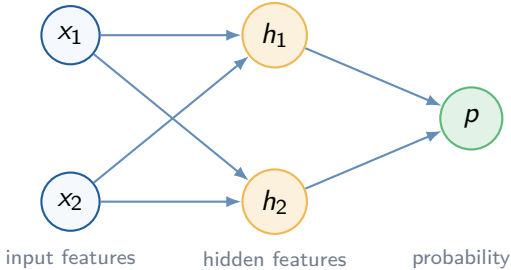


This limitation motivates hidden layers and nonlinear activation functions.

Outline

1. Historical Motivation
2. Perceptron
3. Neural Networks
4. Forward Pass
5. Loss and Training
6. Backward Pass
7. Multi-class Neural Networks
8. Practical considerations
9. Summary

A single linear boundary is not enough for some tasks. A multilayer neural network can **learn useful intermediate features**.



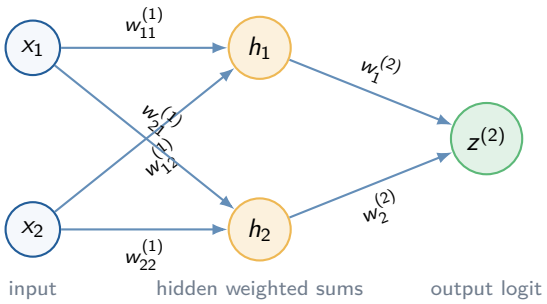
Add a Hidden Layer: More Intermediate Values

Now add two hidden neurons:

$$z_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}$$

$$z_2^{(1)} = w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)}$$

$$z^{(2)} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + b^{(2)}$$



Question

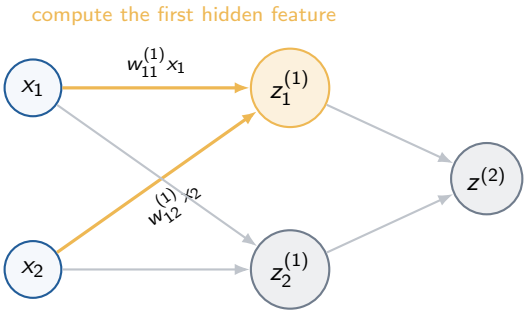
If $h_1 = z_1^{(1)}$ and $h_2 = z_2^{(1)}$, is the model nonlinear?

Hidden Unit 1: A Learned Feature

Focus on the first hidden unit:

$$z_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}$$

It combines the original input features into one intermediate value.

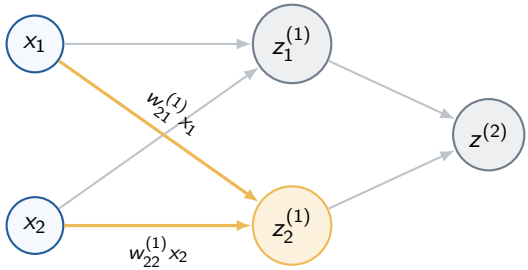


Hidden Unit 2: Another Learned Feature

The second hidden unit learns another combination:

$$z_2^{(1)} = w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)}$$

Different hidden neurons can learn different useful views of the same input.



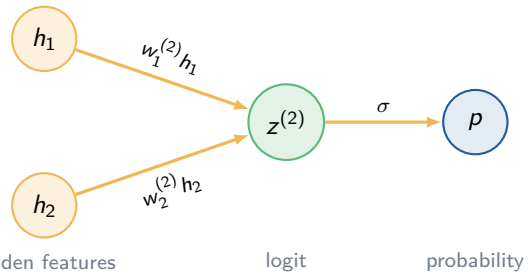
compute the second hidden feature

Output Combines the Hidden Features

The output layer combines the hidden values:

$$z^{(2)} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + b^{(2)}$$

$$p = \sigma(z^{(2)})$$



Interpretation

The output neuron decides how much each learned hidden feature should influence the final prediction.

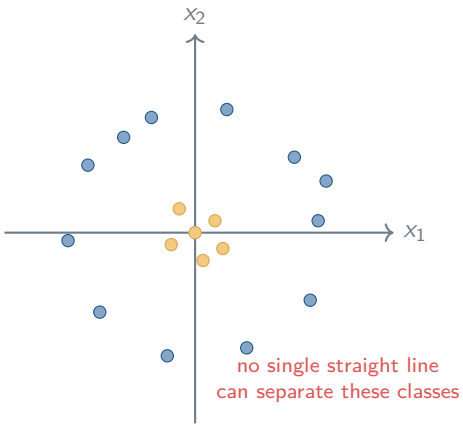
Why Do We Need Nonlinear Functions?

Some data distributions cannot be separated by a single straight line.

- A perceptron learns a **linear** decision boundary.
- Many NLP patterns are nonlinear.

Key idea

A multilayer perceptron uses hidden layers and **nonlinear activation functions** to learn **nonlinear decision boundaries**.

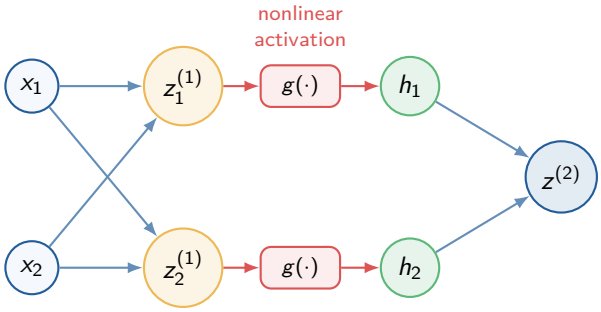


Nonlinear Activation Makes the Hidden Layer Useful

Now insert a nonlinear activation function:

$$h_1 = g(z_1^{(1)}), \qquad h_2 = g(z_2^{(1)})$$

$$z^{(2)} = w_1^{(2)} g(z_1^{(1)}) + w_2^{(2)} g(z_2^{(1)}) + b^{(2)}$$



Common choices:

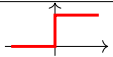
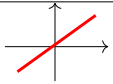
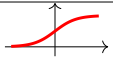
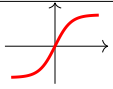
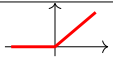
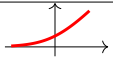
- Sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Rectified Linear Unit:

$$\mathrm{ReLU}(z) = \max(0, z)$$

Common Activation Functions

Activation function	Equation	Example use	1D graph
Unit step	$\phi(z) = \begin{cases} 0, & z < 0, \\ 1, & z \geq 0 \end{cases}$	Perceptron variant	
Sign / Signum	$\phi(z) = \begin{cases} -1, & z < 0, \\ +1, & z \geq 0 \end{cases}$	Bipolar perceptron	
Linear	$\phi(z) = z$	Adaline, linear regression	
Logistic / Sigmoid	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, MLP	
Hyperbolic tangent	$\phi(z) = \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multilayer neural networks	
ReLU	$\phi(z) = \max(0, z)$	Deep neural networks	
Softplus	$\phi(z) = \ln(1 + e^{z})$	Smooth ReLU variant	

NLP example: sentence classifier

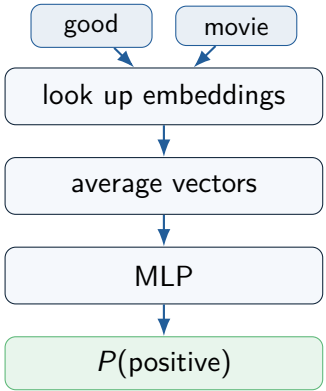
Task: predict whether a short sentence is positive.

Example sentence

good movie

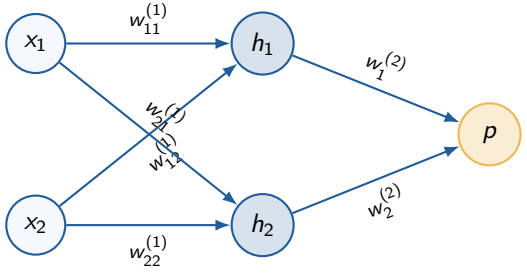
Target label

$y = 1$ means positive sentiment



MLP Architecture Used in This Example

We will use the same small MLP throughout the forward and backward examples.



$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \qquad \boldsymbol{h} = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix}, \qquad p = P(y = 1 \mid \boldsymbol{x})$$

MLP Notation for the 2–2–1 Network

Hidden layer

$$z_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}$$

$$z_2^{(1)} = w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)}$$

$$h_1 = g(z_1^{(1)}), \qquad h_2 = g(z_2^{(1)})$$

Output layer

$$z^{(2)} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + b^{(2)}$$

$$p = \sigma(z^{(2)})$$

Matrix form

$$\boldsymbol{z}^{(1)} = W^{(1)} \boldsymbol{x} + \boldsymbol{b}^{(1)}$$

$$\boldsymbol{h} = g(\boldsymbol{z}^{(1)})$$

$$z^{(2)} = W^{(2)} \boldsymbol{h} + b^{(2)}$$

$$p = \sigma(z^{(2)})$$

Important idea

The hidden layer learns a new representation before the final output is computed.

MLP: Forward Pass and Backward Pass

A multilayer perceptron learns by repeating two main stages.

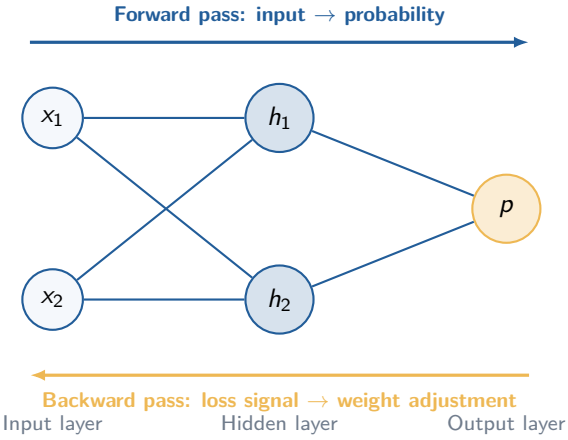
Forward pass

Information flows from the input layer to the output layer. The network uses the current weights to produce a probability.

Backward pass

The loss signal flows backward through the network. This tells the model how the weights should be adjusted.

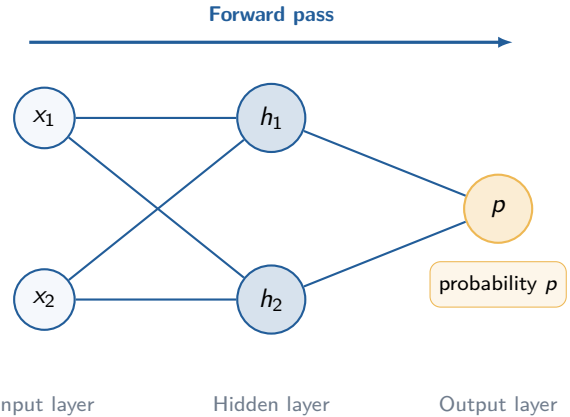
Training idea: repeat forward pass, backward pass, and weight update many times until the model improves.



MLP: Forward Pass

What happens in the forward pass?

- The input features enter the network.
- Each hidden neuron combines the inputs and produces an activation.
- The output neuron uses the hidden activations to produce a probability.
- The probability is then compared with the true label to compute the loss.



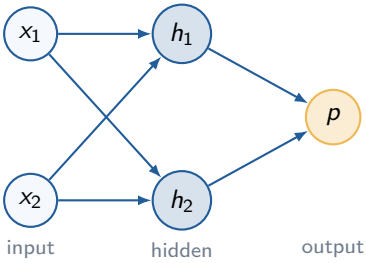
Key idea

The forward pass answers the question: *Given the current weights, what probability does the network assign?*

Outline

1. Historical Motivation
2. Perceptron
3. Neural Networks
4. Forward Pass
5. Loss and Training
6. Backward Pass
7. Multi-class Neural Networks
8. Practical considerations
9. Summary

Forward pass: big picture



Forward pass

Compute values layer by layer, from input to loss.

$$\boldsymbol{x}$$ — input vector

$$\boldsymbol{z}^{(1)} = W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}$$ — hidden pre-activation

$$\boldsymbol{h} = g(\boldsymbol{z}^{(1)})$$ — hidden activation

$$z^{(2)} = W^{(2)}\boldsymbol{h} + b^{(2)}$$ — output logit

$$p = \sigma(z^{(2)})$$ — probability

$$\mathcal{L}(p, y)$$ — loss

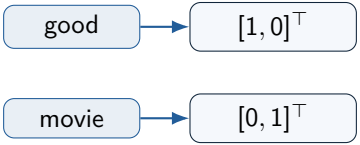
$$\boldsymbol{x} \to \boldsymbol{z}^{(1)} \to \boldsymbol{h} \to z^{(2)} \to p \to \mathcal{L}$$

A forward pass computes the prediction and the loss before any parameter is updated.

Forward pass step 1: word embeddings

For the example *good movie*, use two-dimensional embeddings:

$$\boldsymbol{e}_{\text{good}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \boldsymbol{e}_{\text{movie}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$



Forward pass step 2: sentence representation

A simple sentence vector is the average of word embeddings:

$$\boldsymbol{x} = \frac{\boldsymbol{e}_{\text{good}} + \boldsymbol{e}_{\text{movie}}}{2} = \frac{1}{2}\left(\begin{bmatrix}1\\0\end{bmatrix} + \begin{bmatrix}0\\1\end{bmatrix}\right) = \begin{bmatrix}0.5\\0.5\end{bmatrix}$$

Why this is simplified

Averaging ignores word order, but it lets us focus clearly on neural network computation. Later models add sequence structure.

Forward pass step 3: hidden pre-activation

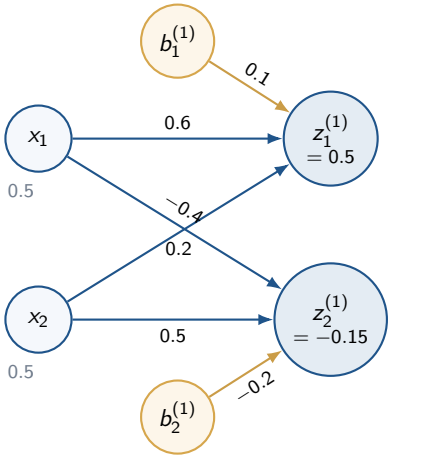
Use a hidden layer with two neurons:

$$W^{(1)} = \begin{bmatrix} 0.6 & 0.2 \\ -0.4 & 0.5 \end{bmatrix}, \qquad \boldsymbol{b}^{(1)} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$\boldsymbol{z}^{(1)} = W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}$$

$$= \begin{bmatrix} 0.6 & 0.2 \\ -0.4 & 0.5 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix} = \begin{bmatrix} 0.5 \\ -0.15 \end{bmatrix}$$

$$z_j^{(1)} = \sum_i w_{ji}^{(1)} x_i + b_j^{(1)}$$



Forward pass step 4: hidden activation

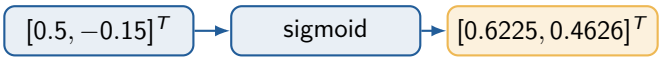
Use the sigmoid activation function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\boldsymbol{h} = \sigma(\boldsymbol{z}^{(1)})$$

$$h_1 = \sigma(0.5) = 0.6225, \qquad h_2 = \sigma(-0.15) = 0.4626$$

$$\boldsymbol{h} = \begin{bmatrix} 0.6225 \\ 0.4626 \end{bmatrix}$$



Forward pass step 5: output score

Use one output neuron:

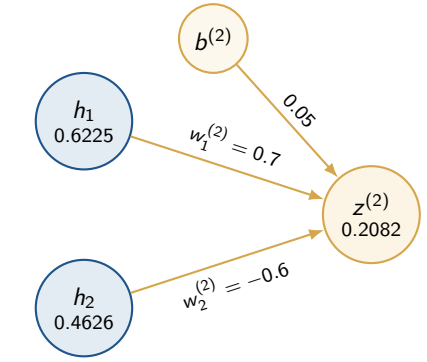
$$W^{(2)} = \begin{bmatrix} 0.7 & -0.6 \end{bmatrix}, \qquad b^{(2)} = 0.05$$

$$z^{(2)} = W^{(2)}\boldsymbol{h} + b^{(2)}$$

$$z^{(2)} = 0.7(0.6225) - 0.6(0.4626) + 0.05$$

$$z^{(2)} = 0.2082$$

The output score is also called a **logit**. A positive logit gives probability above 0.5 after sigmoid.



$$z^{(2)} = \sum_i w_i^{(2)} h_i + b^{(2)}$$

Forward pass step 6: probability and loss

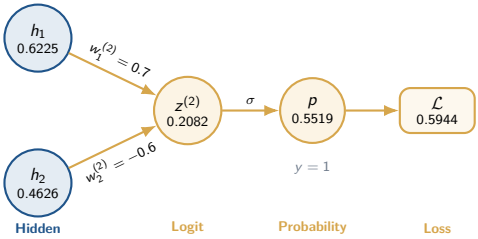
$$p = \sigma(z^{(2)}) = \sigma(0.2082) = 0.5519$$

For binary classification, compute cross-entropy loss:

$$\mathcal{L} = -\left[y \log(p) + (1 - y) \log(1 - p)\right]$$

Here, log denotes the natural logarithm, ln. Since $y = 1$:

$$\mathcal{L} = -\log(0.5519) = 0.5944$$



Interpretation

The prediction is correct but not very confident, so the loss is still moderately high.

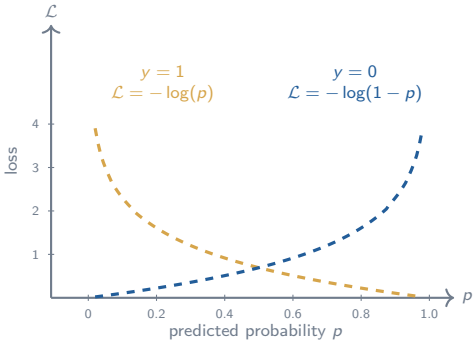
Binary cross-entropy loss: intuition

For binary classification, the loss is:

$$\mathcal{L}(p, y) = -\left[y \log(p) + (1 - y) \log(1 - p)\right]$$

This becomes:

$$\mathcal{L}(p, y) = \begin{cases} -\log(p), & \text{if } y = 1 \\ -\log(1 - p), & \text{if } y = 0 \end{cases}$$



Intuition

- If the true label is $y = 1$, we want p to be close to 1.
- If the true label is $y = 0$, we want p to be close to 0.
- The loss is small when the predicted probability matches the true label, and large when the model is confidently wrong.

Outline

1. Historical Motivation
2. Perceptron
3. Neural Networks
4. Forward Pass
5. Loss and Training
6. Backward Pass
7. Multi-class Neural Networks
8. Practical considerations
9. Summary

Training objective

For a dataset with N examples, the goal is to minimise the average loss:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\left(f_\theta(\boldsymbol{x}_i), y_i\right)$$

where θ denotes all trainable parameters:

$$\theta = \left\{ W^{(1)}, \boldsymbol{b}^{(1)}, W^{(2)}, b^{(2)} \right\}$$

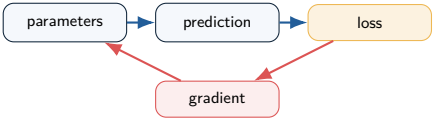
For all trainable parameters together:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$$

For an individual scalar parameter θ_j:

$$\theta_j \leftarrow \theta_j - \eta \frac{\partial \mathcal{L}}{\partial \theta_j}$$

Key idea: Backpropagation computes the gradients; gradient descent uses them to update the parameters.



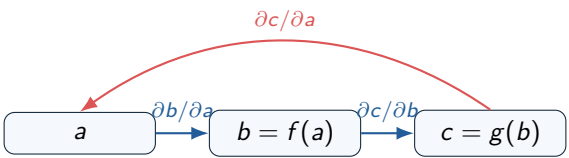
Forward pass computes the prediction and loss; backpropagation computes the gradient.

Chain rule: the core of backpropagation

Scalar chain rule

If a affects b, and b affects c, then

$$\frac{\partial c}{\partial a} = \frac{\partial c}{\partial b}\frac{\partial b}{\partial a}$$



Backpropagation repeatedly applies this rule from the loss backwards to every parameter.

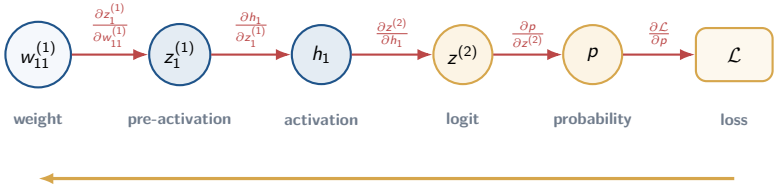
Chain rule in a neural network path

Consider one path from a weight to the loss:

$$w_{11}^{(1)} \to z_1^{(1)} \to h_1 \to z^{(2)} \to p \to \mathcal{L}$$

By the chain rule:

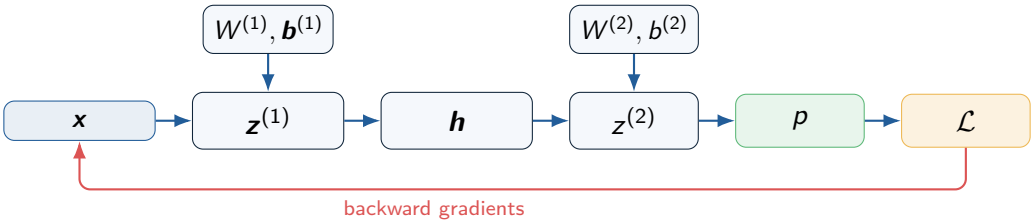
$$\frac{\partial \mathcal{L}}{\partial w_{11}^{(1)}} = \frac{\partial \mathcal{L}}{\partial p} \frac{\partial p}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial h_1} \frac{\partial h_1}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}}$$



Backpropagation idea

Instead of recomputing every full chain separately, we reuse intermediate derivatives and pass them backward through the network.

Computational graph for the numerical example



Forward pass computes values. Backward pass computes how each parameter contributed to the loss.

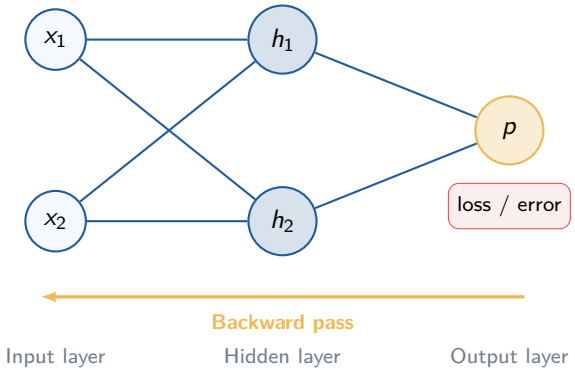
Outline

1. Historical Motivation
2. Perceptron
3. Neural Networks
4. Forward Pass
5. Loss and Training
6. Backward Pass
7. Multi-class Neural Networks
8. Practical considerations
9. Summary

MLP: Backward Pass

What happens in the backward pass?

- The network starts from the error at the output.
- This error is propagated backward through the network.
- Each weight receives a gradient, which tells us how it contributed to the loss.
- These gradients are used to update the weights and improve the model.



Key idea

The backward pass answers the question: *How should the weights change to reduce the loss?*

Backward Pass Step 1a: From Loss to Output Logit

The output part of the network is:

$$z^{(2)} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + b^{(2)}$$

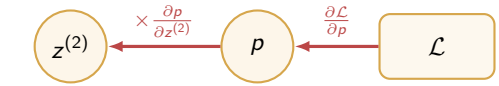
$$p = \sigma(z^{(2)}) = \frac{1}{1 + e^{-z^{(2)}}}$$

$$\mathcal{L} = -\left[y \log(p) + (1 - y) \log(1 - p)\right]$$

Backpropagation applies the chain rule:

$$\frac{\partial \mathcal{L}}{\partial z^{(2)}} = \frac{\partial \mathcal{L}}{\partial p} \frac{\partial p}{\partial z^{(2)}}$$

$$\frac{\partial \mathcal{L}}{\partial z^{(2)}} = p - y$$



$$\frac{\partial \mathcal{L}}{\partial z^{(2)}} = \frac{\partial \mathcal{L}}{\partial p} \frac{\partial p}{\partial z^{(2)}}$$

$$\frac{\partial \mathcal{L}}{\partial p} = -\frac{y}{p} + \frac{1 - y}{1 - p}$$

$$\frac{\partial p}{\partial z^{(2)}} = p(1 - p)$$

Backward Pass Step 1b: Output Error

For sigmoid output with binary cross-entropy, the chain rule simplifies to:

$$\frac{\partial \mathcal{L}}{\partial z^{(2)}} = p - y$$

This quantity is the output-layer error signal:

$$\delta^{(2)} = \frac{\partial \mathcal{L}}{\partial z^{(2)}}$$

From the forward pass:

$$p = 0.5519, \qquad y = 1$$

Therefore:

$$\delta^{(2)} = 0.5519 - 1 = -0.4481$$

Interpretation

$\delta^{(2)}$ tells us how much the loss changes when the output logit $z^{(2)}$ changes. The gradient is negative, which means increasing the output logit $z^{(2)}$ would reduce the loss and increase the predicted probability for the positive class.

Backward Pass Step 2: Output Layer Gradients

The output layer is:

$$z^{(2)} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + b^{(2)}$$

From the output error:

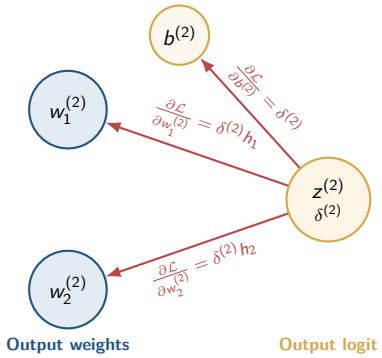
$$\delta^{(2)} = \frac{\partial \mathcal{L}}{\partial z^{(2)}}$$

Therefore:

$$\frac{\partial \mathcal{L}}{\partial w_1^{(2)}} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial w_1^{(2)}} = \delta^{(2)} h_1$$

$$\frac{\partial \mathcal{L}}{\partial w_2^{(2)}} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial w_2^{(2)}} = \delta^{(2)} h_2$$

$$\frac{\partial \mathcal{L}}{\partial b^{(2)}} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial b^{(2)}} = \delta^{(2)}(1)$$



Why this step matters

These values are not used to compute the hidden-layer gradient. They are stored and later used to update the output parameters $W^{(2)}$ and $b^{(2)}$.

Backward Pass Step 2: Numerical Output Layer Gradients

The output layer is:

$$z^{(2)} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + b^{(2)}$$

From the output error:

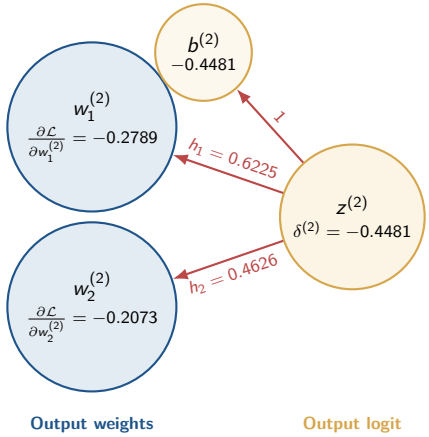
$$\delta^{(2)} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} = -0.4481$$

Therefore:

$$\frac{\partial \mathcal{L}}{\partial w_1^{(2)}} = \delta^{(2)} h_1 \quad = -0.4481(0.6225) = -0.2789$$

$$\frac{\partial \mathcal{L}}{\partial w_2^{(2)}} = \delta^{(2)} h_2 \quad = -0.4481(0.4626) = -0.2073$$

$$\frac{\partial \mathcal{L}}{\partial b^{(2)}} = \delta^{(2)}(1) \quad = -0.4481$$



Backward Pass Step 3: Send Gradient to Hidden Units

The output logit is:

$$z^{(2)} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + b^{(2)}$$

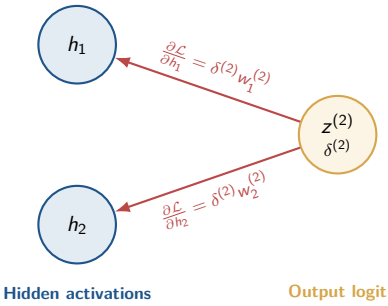
From the output error:

$$\delta^{(2)} = \frac{\partial \mathcal{L}}{\partial z^{(2)}}$$

Therefore:

$$\frac{\partial \mathcal{L}}{\partial h_1} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial h_1} = \delta^{(2)} w_1^{(2)}$$

$$\frac{\partial \mathcal{L}}{\partial h_2} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial h_2} = \delta^{(2)} w_2^{(2)}$$



Key idea

The gradient is sent backward from the output logit to each hidden activation.

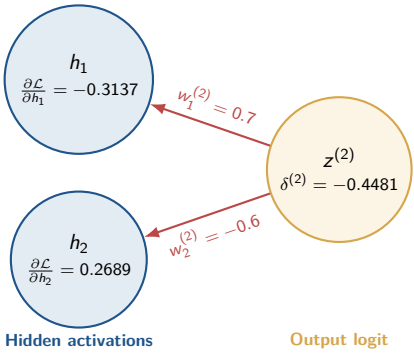
Backward Pass Step 3: Numerical Hidden Gradients

Using:

$$\delta^{(2)} = -0.4481, \qquad w_1^{(2)} = 0.7, \qquad w_2^{(2)} = -0.6$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial h_1} &= \delta^{(2)} w_1^{(2)} \\ &= -0.4481(0.7) \\ &= -0.3137\end{aligned}$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial h_2} &= \delta^{(2)} w_2^{(2)} \\ &= -0.4481(-0.6) \\ &= 0.2689\end{aligned}$$



Backward Pass Step 4: Through the Sigmoid Hidden Layer

For each hidden neuron:

$$h_i = \sigma(z_i^{(1)}) = \frac{1}{1 + e^{-z_i^{(1)}}}$$

$$\frac{\partial h_i}{\partial z_i^{(1)}} = h_i(1 - h_i)$$

Therefore:

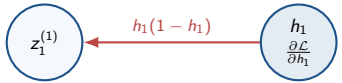
$$\delta_i^{(1)} = \frac{\partial \mathcal{L}}{\partial z_i^{(1)}} = \frac{\partial \mathcal{L}}{\partial h_i} \frac{\partial h_i}{\partial z_i^{(1)}}$$

For hidden neuron 1:

$$\delta_1^{(1)} = \frac{\partial \mathcal{L}}{\partial h_1} h_1(1 - h_1)$$

For hidden neuron 2:

$$\delta_2^{(1)} = \frac{\partial \mathcal{L}}{\partial h_2} h_2(1 - h_2)$$



Key idea

The sigmoid derivative converts the gradient with respect to h_i into the gradient with respect to $z_i^{(1)}$.

Backward Pass Step 4: Numerical Hidden Error Signals

Using:

$$h_1 = 0.6225, \qquad h_2 = 0.4626$$

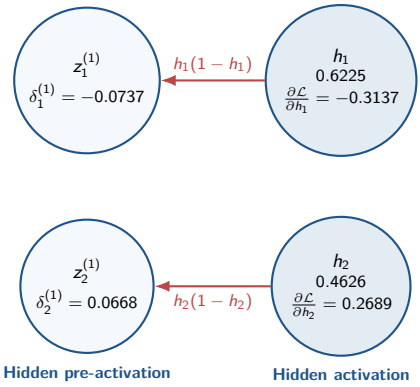
$$\frac{\partial \mathcal{L}}{\partial h_1} = -0.3137, \qquad \frac{\partial \mathcal{L}}{\partial h_2} = 0.2689$$

For hidden neuron 1:

$$\begin{aligned}\delta_1^{(1)} &= \frac{\partial \mathcal{L}}{\partial h_1} h_1(1-h_1) \\ &= -0.3137(0.6225)(1-0.6225) \\ &= -0.0737\end{aligned}$$

For hidden neuron 2:

$$\begin{aligned}\delta_2^{(1)} &= \frac{\partial \mathcal{L}}{\partial h_2} h_2(1-h_2) \\ &= 0.2689(0.4626)(1-0.4626) \\ &= 0.0668\end{aligned}$$



Backward Pass Step 5: Hidden Layer Gradients

The hidden pre-activations are:

$$z_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}$$
$$z_2^{(1)} = w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)}$$

For hidden neuron 1:

$$\frac{\partial \mathcal{L}}{\partial w_{11}^{(1)}} = \frac{\partial \mathcal{L}}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}} = \delta_1^{(1)} x_1$$

$$\frac{\partial \mathcal{L}}{\partial w_{12}^{(1)}} = \frac{\partial \mathcal{L}}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{12}^{(1)}} = \delta_1^{(1)} x_2$$

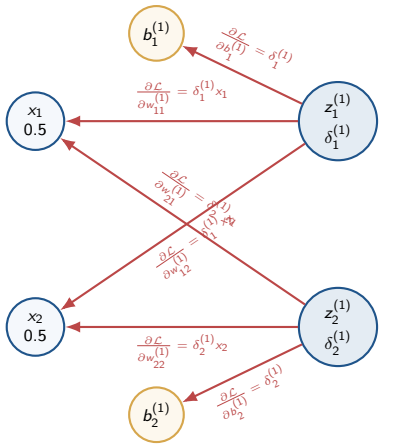
$$\frac{\partial \mathcal{L}}{\partial b_1^{(1)}} = \frac{\partial \mathcal{L}}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial b_1^{(1)}} = \delta_1^{(1)}$$

For hidden neuron 2:

$$\frac{\partial \mathcal{L}}{\partial w_{21}^{(1)}} = \frac{\partial \mathcal{L}}{\partial z_2^{(1)}} \frac{\partial z_2^{(1)}}{\partial w_{21}^{(1)}} = \delta_2^{(1)} x_1$$

$$\frac{\partial \mathcal{L}}{\partial w_{22}^{(1)}} = \frac{\partial \mathcal{L}}{\partial z_2^{(1)}} \frac{\partial z_2^{(1)}}{\partial w_{22}^{(1)}} = \delta_2^{(1)} x_2$$

$$\frac{\partial \mathcal{L}}{\partial b_2^{(1)}} = \frac{\partial \mathcal{L}}{\partial z_2^{(1)}} \frac{\partial z_2^{(1)}}{\partial b_2^{(1)}} = \delta_2^{(1)}$$



Backward Pass Step 5: Numerical Hidden Layer Gradients

Using:

$$x_1 = 0.5, \quad x_2 = 0.5, \quad \delta_1^{(1)} = -0.0737, \quad \delta_2^{(1)} = 0.0668$$

For hidden neuron 1:

$$\frac{\partial \mathcal{L}}{\partial w_{11}^{(1)}} = \delta_1^{(1)} x_1 = -0.0737(0.5) = -0.0369$$

$$\frac{\partial \mathcal{L}}{\partial w_{12}^{(1)}} = \delta_1^{(1)} x_2 = -0.0737(0.5) = -0.0369$$

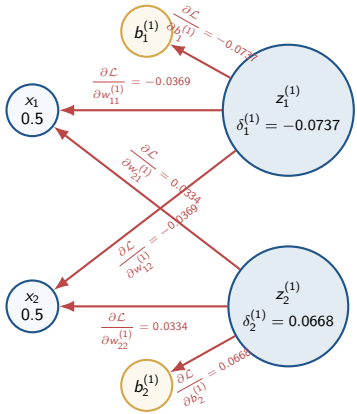
$$\frac{\partial \mathcal{L}}{\partial b_1^{(1)}} = \delta_1^{(1)} = -0.0737$$

For hidden neuron 2:

$$\frac{\partial \mathcal{L}}{\partial w_{21}^{(1)}} = \delta_2^{(1)} x_1 = 0.0668(0.5) = 0.0334$$

$$\frac{\partial \mathcal{L}}{\partial w_{22}^{(1)}} = \delta_2^{(1)} x_2 = 0.0668(0.5) = 0.0334$$

$$\frac{\partial \mathcal{L}}{\partial b_2^{(1)}} = \delta_2^{(1)} = 0.0668$$



Each numerical gradient is obtained from hidden error × input, or from the bias link.

Backward Pass Step 6: Gradient Descent Update

Backpropagation has now computed the gradients. The next step is to update the model parameters.

For all trainable parameters:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$$

where

$$\theta = \left\{ W^{(1)}, \boldsymbol{b}^{(1)}, W^{(2)}, b^{(2)} \right\}$$

For an individual scalar parameter θ_j:

$$\theta_j \leftarrow \theta_j - \eta \frac{\partial \mathcal{L}}{\partial \theta_j}$$

In this numerical example:

$$\eta = 0.1$$

Key idea

new parameter = old parameter − learning rate × gradient

Backward Pass Step 6a: Output Weight Update

Gradient descent rule:

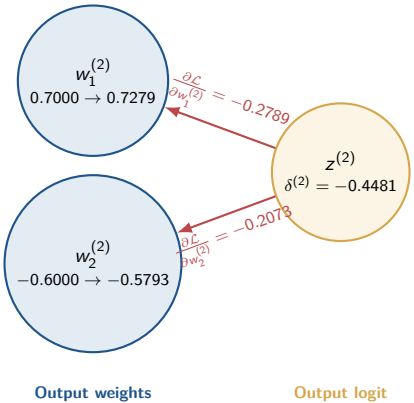
$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial \mathcal{L}}{\partial w}, \qquad \eta = 0.1$$

For $w_1^{(2)}$:

$$\begin{aligned} w_{1,\text{new}}^{(2)} &= w_{1,\text{old}}^{(2)} - 0.1 \frac{\partial \mathcal{L}}{\partial w_1^{(2)}} \\ &= 0.7000 - 0.1(-0.2789) \\ &= 0.7279 \end{aligned}$$

For $w_2^{(2)}$:

$$\begin{aligned} w_{2,\text{new}}^{(2)} &= w_{2,\text{old}}^{(2)} - 0.1 \frac{\partial \mathcal{L}}{\partial w_2^{(2)}} \\ &= -0.6000 - 0.1(-0.2073) \\ &= -0.5793 \end{aligned}$$



Key idea

A negative gradient makes the corresponding weight increase after gradient descent.

Backward Pass Step 6b: Output Bias Update

From the previous step:

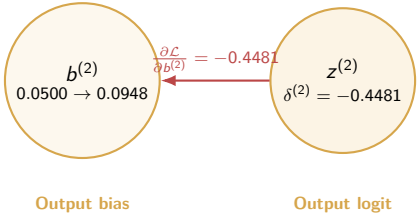
$$\frac{\partial \mathcal{L}}{\partial b^{(2)}} = -0.4481$$

Old output bias:

$$b^{(2)}_{\text{old}} = 0.0500$$

Gradient descent update:

$$\begin{aligned} b^{(2)}_{\text{new}} &= b^{(2)}_{\text{old}} - 0.1 \frac{\partial \mathcal{L}}{\partial b^{(2)}} \\ &= 0.0500 - 0.1(-0.4481) \\ &= 0.0500 + 0.0448 \\ &= 0.0948 \end{aligned}$$



Backward Pass Step 7a: Hidden Weight Update

Weights into hidden neuron 1:

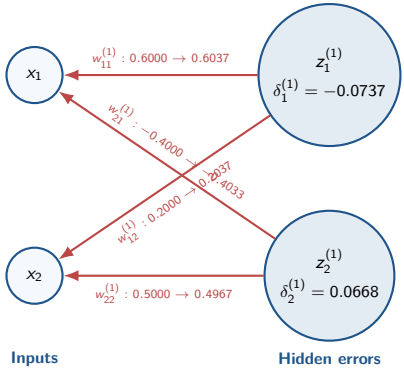
$$w_{11,\text{new}}^{(1)} = w_{11,\text{old}}^{(1)} - 0.1\frac{\partial \mathcal{L}}{\partial w_{11}^{(1)}}$$
$$= 0.6000 - 0.1(-0.0369) = 0.6037$$

$$w_{12,\text{new}}^{(1)} = w_{12,\text{old}}^{(1)} - 0.1\frac{\partial \mathcal{L}}{\partial w_{12}^{(1)}}$$
$$= 0.2000 - 0.1(-0.0369) = 0.2037$$

Weights into hidden neuron 2:

$$w_{21,\text{new}}^{(1)} = w_{21,\text{old}}^{(1)} - 0.1\frac{\partial \mathcal{L}}{\partial w_{21}^{(1)}}$$
$$= -0.4000 - 0.1(0.0334) = -0.4033$$

$$w_{22,\text{new}}^{(1)} = w_{22,\text{old}}^{(1)} - 0.1\frac{\partial \mathcal{L}}{\partial w_{22}^{(1)}}$$
$$= 0.5000 - 0.1(0.0334) = 0.4967$$



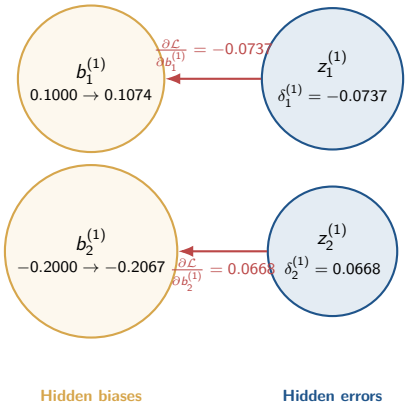
Backward Pass Step 7b: Hidden Bias Update

For hidden bias $b_1^{(1)}$:

$$\begin{aligned} b_{1,\text{new}}^{(1)} &= b_{1,\text{old}}^{(1)} - 0.1 \frac{\partial \mathcal{L}}{\partial b_1^{(1)}} \\ &= 0.1000 - 0.1(-0.0737) \\ &= 0.1074 \end{aligned}$$

For hidden bias $b_2^{(1)}$:

$$\begin{aligned} b_{2,\text{new}}^{(1)} &= b_{2,\text{old}}^{(1)} - 0.1 \frac{\partial \mathcal{L}}{\partial b_2^{(1)}} \\ &= -0.2000 - 0.1(0.0668) \\ &= -0.2067 \end{aligned}$$



Backward Pass Step 7c: Updated Parameters and Check

After one gradient-descent update:

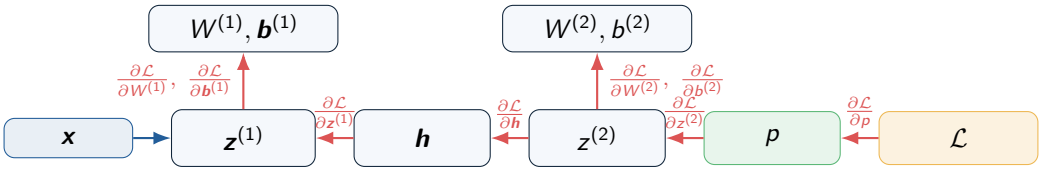
Parameter	Old value	New value
$w_1^{(2)}$	0.7000	0.7279
$w_2^{(2)}$	-0.6000	-0.5793
$b^{(2)}$	0.0500	0.0948
$w_{11}^{(1)}$	0.6000	0.6037
$w_{12}^{(1)}$	0.2000	0.2037
$w_{21}^{(1)}$	-0.4000	-0.4033
$w_{22}^{(1)}$	0.5000	0.4967
$b_1^{(1)}$	0.1000	0.1074
$b_2^{(1)}$	-0.2000	-0.2067

Check

After one update, the probability increases and the loss decreases:

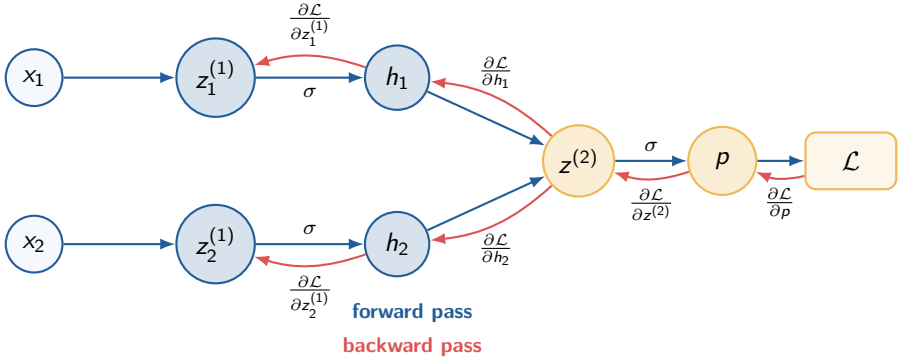
$$p : 0.5519 \to 0.5704, \qquad \mathcal{L} : 0.5944 \to 0.5614.$$

Backward pass summary



Backpropagation computes gradients along the path from the loss back to earlier quantities. The optimizer then uses the parameter gradients to update the model.

Backpropagation: Big Picture



Forward: $\boldsymbol{x} \to \boldsymbol{z}^{(1)} \to \boldsymbol{h} \to z^{(2)} \to p \to \mathcal{L}$

Backward: $\mathcal{L} \to p \to z^{(2)} \to \boldsymbol{h} \to \boldsymbol{z}^{(1)}$

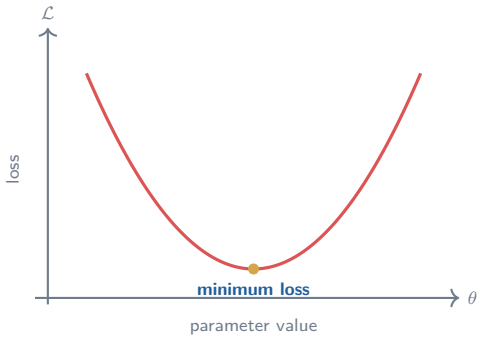
Reducing loss with gradient descent

Gradient descent tries to find parameters that give a smaller loss.

For one parameter θ, we can imagine plotting:

$$\mathcal{L}(\theta)$$

- The vertical axis is the loss.
- The horizontal axis is the parameter value.
- The goal is to move towards a minimum.



Key idea

Instead of checking every possible value of θ, gradient descent uses the slope to decide where to move next.

Learning rate: too small

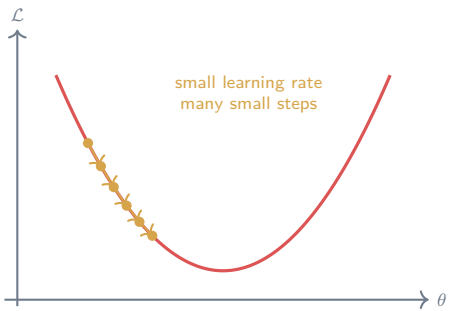
The learning rate η controls the size of the update.

If η is too small:

- each step is tiny,
- training is stable,
- but convergence can be very slow.

Problem

The model may need many epochs before the loss becomes small.



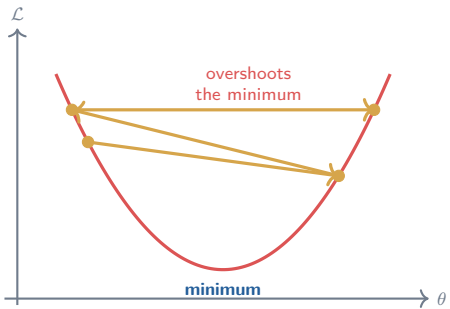
Learning rate: too large

If η is too large:

- updates become too aggressive,
- the parameter may overshoot the minimum,
- training can oscillate or diverge.

Problem

A large learning rate can increase the loss instead of reducing it.



Learning rate: reasonable step size

A reasonable learning rate gives useful progress without overshooting.

- Steps are large enough to learn efficiently.
- Steps become smaller near the minimum because the gradient becomes smaller.
- The loss gradually decreases.



Practical note

Learning rate is a hyperparameter and often needs tuning.

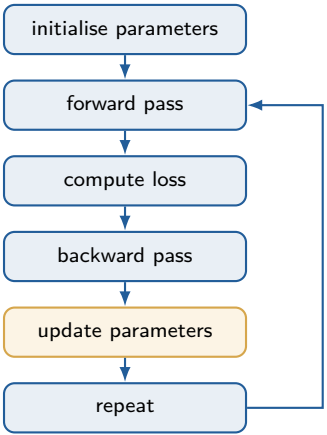
Gradient descent algorithm

For neural networks, θ represents all trainable parameters:

$$\theta = \{W^{(1)}, \boldsymbol{b}^{(1)}, W^{(2)}, b^{(2)}, \ldots\}$$

Algorithm

1. Initialise parameters.
2. Compute prediction by forward pass.
3. Compute loss.
4. Compute gradients by backward pass.
5. Update parameters using an optimiser (e.g. gradient descent).
6. Repeat until the loss is sufficiently small.

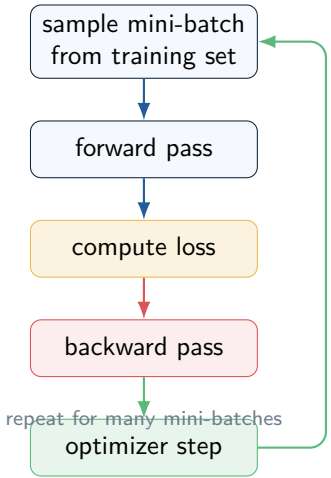


Mini-batch training loop

Example sentiment dataset

Split	Example texts	Label
Training set	“good movie”, “very bad plot”, “not interesting”, “really nice”	pos / neg
Validation set	“very good”, “quite boring”	pos / neg
Test set	“excellent acting”, “waste of time”	pos / neg

- **Training set:** used to compute gradients and update parameters.
- **Validation set:** used to tune hyperparameters and monitor overfitting.
- **Test set:** used only for the final evaluation.



Mini-batch idea

At each iteration, we take a small batch from the **training set** rather than using the entire dataset at once.

One full pass through the training set is called an **epoch**.

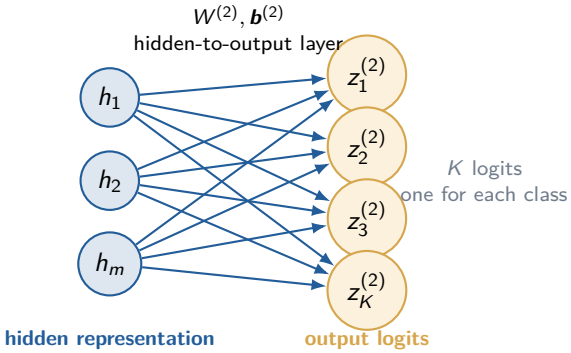
Outline

1. Historical Motivation
2. Perceptron
3. Neural Networks
4. Forward Pass
5. Loss and Training
6. Backward Pass
7. Multi-class Neural Networks
8. Practical considerations
9. Summary

From binary sentiment to multi-class NLP

Many NLP tasks have more than two labels:

- sentiment: negative, neutral, positive;
- topic classification: sports, finance, politics, technology;
- intent classification: greeting, booking, complaint, payment;
- part-of-speech tagging: noun, verb, adjective, adverb.



For K classes, the output layer produces K logits:

$$\boldsymbol{z}^{(2)} = W^{(2)}\boldsymbol{h} + \boldsymbol{b}^{(2)} \in \mathbb{R}^K$$

Here, $^{(2)}$ means the output layer because the hidden layer is layer $^{(1)}$.

Softmax output

Softmax converts logits into a probability distribution over K classes:

$$p_k = \frac{\exp(z_k^{(2)})}{\sum_{j=1}^{K} \exp(z_j^{(2)})}$$

Example

Suppose the logits for three sentiment classes are:

$$\boldsymbol{z}^{(2)} = [1.2,\ 0.3,\ -0.8]^\top$$

Compute the exponentials:

$$e^{1.2} = 3.3201, \qquad e^{0.3} = 1.3499, \qquad e^{-0.8} = 0.4493$$

Compute the denominator:

$$\sum_j e^{z_j^{(2)}} = 3.3201 + 1.3499 + 0.4493 = 5.1193$$

Softmax output: calculation

Using the denominator:

$$\sum_j e^{z_j^{(2)}} = 5.1193$$

The softmax probabilities are:

$$\boldsymbol{p} = \left[\frac{3.3201}{5.1193}, \frac{1.3499}{5.1193}, \frac{0.4493}{5.1193}\right]$$

$$= [0.6485,\ 0.2637,\ 0.0878]$$

Check:

$$0.6485 + 0.2637 + 0.0878 = 1.0000$$

Key idea

Softmax turns raw scores into class probabilities that sum to 1. The predicted class is $\hat{y} = \arg\max_k p_k$.

Cross-entropy loss

For a one-hot target vector $\boldsymbol{y}$, the multi-class cross-entropy is:

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log(p_k)$$

If the correct class is c, this becomes:

$$\mathcal{L} = -\log(p_c)$$

Practical meaning

The model is penalised when it assigns low probability to the correct class.

Cross-entropy only keeps the correct class

Because $\boldsymbol{y}$ is one-hot, only the correct class has value 1:

$$\boldsymbol{y} = [0,\ 0,\ 1]$$

So all incorrect-class terms disappear:

$$\mathcal{L} = -\left[0 \log(p_1) + 0 \log(p_2) + 1 \log(p_3)\right]$$

$$= -\log(p_3)$$

In general, if the correct class is c:

$$\mathcal{L} = -\log(p_c)$$

Key idea

Cross-entropy mainly checks how much probability the model assigns to the correct class.

Outline

1. Historical Motivation
2. Perceptron
3. Neural Networks
4. Forward Pass
5. Loss and Training
6. Backward Pass
7. Multi-class Neural Networks
8. Practical considerations
9. Summary

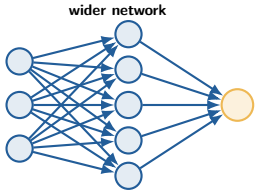
Breadth and depth of a neural network

Breadth

Breadth means the **number of neurons in a layer**.

wider layer = more hidden units

A wider layer can learn more feature combinations in parallel.

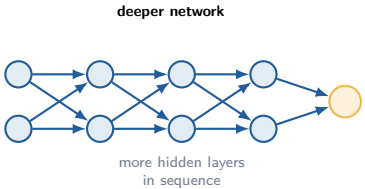


Depth

Depth means the **number of hidden layers**.

deeper network = more transformations

A deeper network can build more hierarchical representations.



Breadth vs depth: practical choice

When to increase breadth

Use more hidden units when:

- the task needs many feature combinations;
- the dataset is large enough;
- the model is underfitting.

When to increase depth

Use more layers when:

- the task needs hierarchical abstraction;
- simple features must be combined repeatedly;
- the model needs more expressive transformations.

But more is not always better

Increasing breadth or depth:

- increases computation;
- increases risk of overfitting;
- may require regularisation;
- may require more training data.

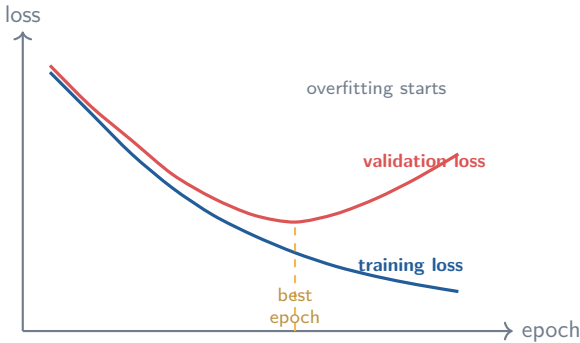
Overfitting in NLP models

Overfitting symptoms

- training loss keeps decreasing;
- validation loss starts increasing;
- model memorises training phrases;
- poor generalisation to new text.

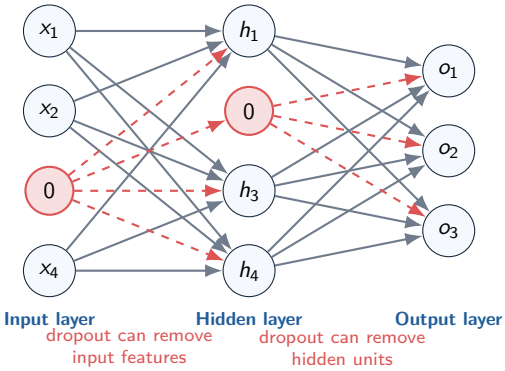
Common remedies

- more data or augmentation;
- dropout;
- weight decay;
- early stopping;
- smaller model.



Dropout intuition

- During training, dropout **randomly sets some units to zero**.
- This can happen to **input features** or **hidden units**.
- The dropped units do not contribute to the forward pass for that training step.
- This prevents the network from depending too much on a few specific features.



Key idea

Each mini-batch sees a slightly different sub-network, which encourages the model to learn more robust representations.

At test time, dropout is turned off, and the full network is used.

What MLPs still miss in NLP

If we average embeddings, these two sentences can look similar:

Same words, different order

dog bites man

man bites dog

Limitation: A basic MLP over averaged embeddings has no explicit memory of word order.

Consequence: For sequence-sensitive tasks, we need architectures that model position and context.

Outline

1. Historical Motivation
2. Perceptron
3. Neural Networks
4. Forward Pass
5. Loss and Training
6. Backward Pass
7. Multi-class Neural Networks
8. Practical considerations
9. Summary

Lecture summary

- A perceptron is a linear classifier built from weighted inputs, a bias and a decision function.
- An MLP uses hidden layers and nonlinear activations to learn intermediate representations.
- The forward pass computes logits, probabilities and loss.
- The backward pass uses the chain rule to compute gradients efficiently.
- In NLP, neural networks can learn embeddings and hidden features useful for text classification.

References

- Frank Rosenblatt. The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, 1958.
- Marvin Minsky and Seymour Papert. *Perceptrons*. MIT Press, 1969.
- David E. Rumelhart, Geoffrey E. Hinton and Ronald J. Williams. Learning Representations by Back-propagating Errors. *Nature*, 1986.
- Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- Ian Goodfellow, Yoshua Bengio and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- Michael A. Nielsen. *Neural Networks and Deep Learning*. 2015.
- Nitish Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 2014.
- Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. 3rd edition draft.